QuickReview: a multi-pass automated code reviewer,
its design, its cost, and how it performs on a verified defect benchmark

Mateusz Jakubczak
Quickchat AI
mateusz@quickchat.ai

Draft of 2026-10-04

Abstract

QuickReview is an automated pull-request reviewer designed for recall: to find as many of a change's real problems as possible, the serious ones first. It rests on a bet that a review assembled from many narrow, independent reads of a change, each checked against the code before it is posted, finds more of them than one long read does. For each PR one session plans about ten lens passes from the change by explicit rules — a baseline, a tests lens, a line-by-line sweep over every changed file, four code-quality lenses and narrow sub-lenses triggered by what the diff or the PR text says — runs each as an isolated read-only agent, merges what they raise, and has a validator re-derive every surviving candidate from the code before one review is composed. We measure it on a verified defect benchmark over the 185 runnable PRs of AACR-Bench, built by the same authors in a companion paper. Its frozen build finds 97% of the 90 high-severity defects (87.5, a partial find counting half), 94.5% of the high- and medium-severity ones and 89% of all 520, ahead of every other reviewer at every severity; the strongest third-party configuration, pi-review's three-reviewer panel, finds 89%, 85% and 67%. On two external benchmarks it has the highest recall under one judge: 87% of the golden comments of Martian's offline benchmark that an audit judged real, and 52% of the functional bugs of Qodo's PR-Review-Bench (45% of all its issues, second to 46%), ahead on both of the same three third-party harnesses run on the same model (they find 53–78% of the real goldens and 44–46% of the functional bugs). Recall is bought with comments, money and time. It posts 6.9 comments per PR, 92% of them correct but 42% minor, which gives it the lowest Martian F1 and the fourth-highest Qodo F1; it costs $2.06 per PR at list prices (median $1.87; $2.84 and $2.48 on Martian's and Qodo's larger PRs), 51% more than the build of the main comparison and 4 to 31 times the third-party tools; and it takes a median 17.5 minutes per PR, 1.8 times that build, against 0.7–2.3 minutes for the others. The design is studied on that earlier build, which finds 79% (95% interval 75–82%) at $1.36 per PR, ahead of the panel's 67% almost entirely on low-severity defects. Most of its spend is in redundant lenses, whose value is very uneven: the per-file sweep alone found 29 defects no other lens raised, while two code-quality lenses cost a tenth of the budget for one such defect. With the design fixed, the base model moves recall from 28% to 84%; no equal-budget alternative was run. Because the authors built the benchmark partly from this reviewer's own earlier output, we state where that could inflate these numbers.

Contents

1 Introduction **3**

2 Related work **4**

3 Design **5**
3.1 Principles 5
3.2 One round 5
3.3 The lens catalogue 6

3.4 The validator and its dispositions . . . 8
3.5 Models, tiers and presets . . . 8
3.6 What the benchmark measures, and what it leaves out . . . 8

4 Evaluation setup **9**

5 Results (RQ1) **10**
5.1 Defects found . . . 10
5.2 What the comments are . . . 11
5.3 Where the lead comes from . . . 12
5.4 AACR-Bench's own metric . . . 12

6 Cost: where it goes and what it buys (RQ2) **13**
6.1 Where the money goes . . . 13
6.2 What the compute buys . . . 14

7 The closest competitor: pi-review's panel (RQ3) **15**
7.1 Two designs . . . 15
7.2 Results on all 185 PRs . . . 15
7.3 Why QuickReview finds more: candidate explanations . . . 17
7.4 Design or budget? . . . 17

8 Base model (RQ4) **17**

9 Stability (RQ5) **19**

10 The frozen build **20**

11 What these numbers do not show **21**

12 Conclusion **22**

A Tuning history in brief **25**

B Chronology of result-affecting decisions **25**

1 Introduction

A code reviewer, human or automated, is asked to find the problems a change introduces before it is merged. Automated reviewers built on large language models now ship from tool vendors, open-source projects and model providers [4, 9, 3, 8, 7, 5], and most of them share one shape: an agent reads the diff, explores the repository as it sees fit, and writes its findings in one pass. That shape has a characteristic way of failing. When we analysed the defects our own reviewer kept missing during development (48 real defects across 30 PRs that several runs failed to find; Appendix A), most were never raised by any part of the review at all, not raised and then discarded. Fifteen were attention failures: the code was in front of the model and it read past a misspelled name or an unchecked identifier. Eleven needed knowledge of what a third-party library actually does, which a reviewer that never installs anything cannot look up. Several needed a procedure applied exhaustively — every consumer of a widened check, every pairing of old and new versions — that a general read does not apply.

QuickReview[1] is our answer to those failures. It runs every day on the authors' production repository: a daemon receives pull-request events and, for every PR a team asks it to review, one session writes one review with inline comments, a verdict, and a section of points for a human to double-check. Its design rests on three choices. A review is the union of many narrow, independent passes rather than one long read, so that a defect one pass misses can be raised by another. Every candidate is re-derived from the code by a separate validator before anything is posted, so that the volume of candidates does not become volume of noise. And the reviewer only reads code, CI results and library sources; it never executes anything. Those choices cost compute, and the question this report answers is what they buy for it.

Recall first. Because the design aims at finding as many real problems as possible, we lead with recall — of high-severity defects first, then of all defects — and report precision, comment accuracy, cost and time beside it as the price of that aim. Table 16 puts the shipping build's recall, cost and time next to the strongest competitors'.

Research questions.

RQ1 How many verified defects does QuickReview find, how accurate is what it says, and how does it compare with the review tools a team could adopt instead, each at its best configuration and on the same model? (Section 5)

RQ2 What does it cost, where does the cost go, and what does each part of the design contribute? (Section 6)

RQ3 How does it compare with the third-party design closest to it, pi-review's three-reviewer panel, and what can that comparison say about design versus budget? (Section 7)

RQ4 How much of the result is the model under the design? (Section 8)

RQ5 How stable is the result across runs? (Section 9)

Contributions. (1) A precise description of a multi-pass review design: its lens catalogue and the rules that plan each lens, its validator and the dispositions it can give a candidate, and its model tiers and presets (Section 3). (2) Its evaluation against seven other reviewer configurations on a verified defect benchmark, with intervals, severity, kind and size breakdowns and the benchmark's own metric. (3) A decomposition of its cost into lenses, validator and session, with per-lens attribution showing which redundancy pays and which does not. (4) A comparison with the closest competitor design on all 185 PRs, separating what the data show from what they only suggest. (5) A base-model study and two stability checks.

[1] Its code, configuration and earlier reports call it `quickreviewd`.

Conflict of interest. The authors built QuickReview, and the same team, using agents directed by the same assistant that designed the reviewer's tuning levers, built the benchmark's answer key — partly from candidate defects in QuickReview's own earlier reviews — and wrote the judge's prompt. The companion paper [1] measures how far this could inflate the results; Section 11 repeats what was measured and what was not. We report every number we have, including those that do not favour QuickReview.

2 Related work

Human code review. Studies of modern code review find that it is valued for knowledge transfer and design discussion as much as for defects, and that most of what reviewers raise concerns maintainability rather than functional bugs [10, 11, 12]. QuickReview's code-quality lenses follow that finding: they look for what will cost the next reader, not only for what breaks. Its lens structure has an older ancestor in software inspection: perspective-based reading assigns each inspector one stakeholder's perspective and a procedure to follow, and reported higher team defect detection than with the reading technique it replaced [13]. A QuickReview lens is a perspective with a procedure, executed by a model.

LLM code review in practice and research. Industrial reports describe LLM reviewers deployed at scale: Google's AutoCommenter flags violations of coding best practices in review [14], and a study of an LLM review tool in industry found that developers acted on many of its comments while noting noise and longer review times [15]. Research systems range from models pre-trained for review tasks [16] to multi-agent designs in which agents with different roles discuss a change [17]. The tools compared here are agents that read the diff and the repository, with one exception that is itself multi-angle: pi-review's panel runs three specialists and keeps a finding two of them agree on [7]. Benchmarks for these systems, and why we built a verified defect benchmark rather than reuse reference comments, are discussed in the companion paper [1].

Redundancy, ensembles and verification. Sampling a model several times and aggregating the answers improves accuracy on reasoning tasks [18], more sampled agents with majority voting keep helping as their number grows [19], and coverage — the chance that at least one of many samples is right — grows steadily with the number of samples [20]. Coverage needs a way to pick what is right: trained verifiers [21], a model answering its own verification questions [22], or several models debating to a common answer [23]. Self-correction without external feedback is unreliable [24]; QuickReview's validator is meant to get its feedback from the code, re-deriving each claim by reading it. Two differences from this literature matter. QuickReview's passes are not samples of one prompt but different prompts, so they aim at different defects as well as at the same one; and their errors are not independent — the classic result for independently written program versions is that faults coincide far more often than independence predicts [25], which bounds what redundancy can buy.

Cost-aware evaluation. Agent benchmarks that report accuracy alone reward spending; cost belongs beside accuracy, and comparisons should be read on the cost–accuracy frontier [26]. Routing work to cheaper models where they suffice can cut cost at equal quality [27]. We report cost per PR, per defect found and per additional defect, and we compare against alternatives that are cheaper, not only better.

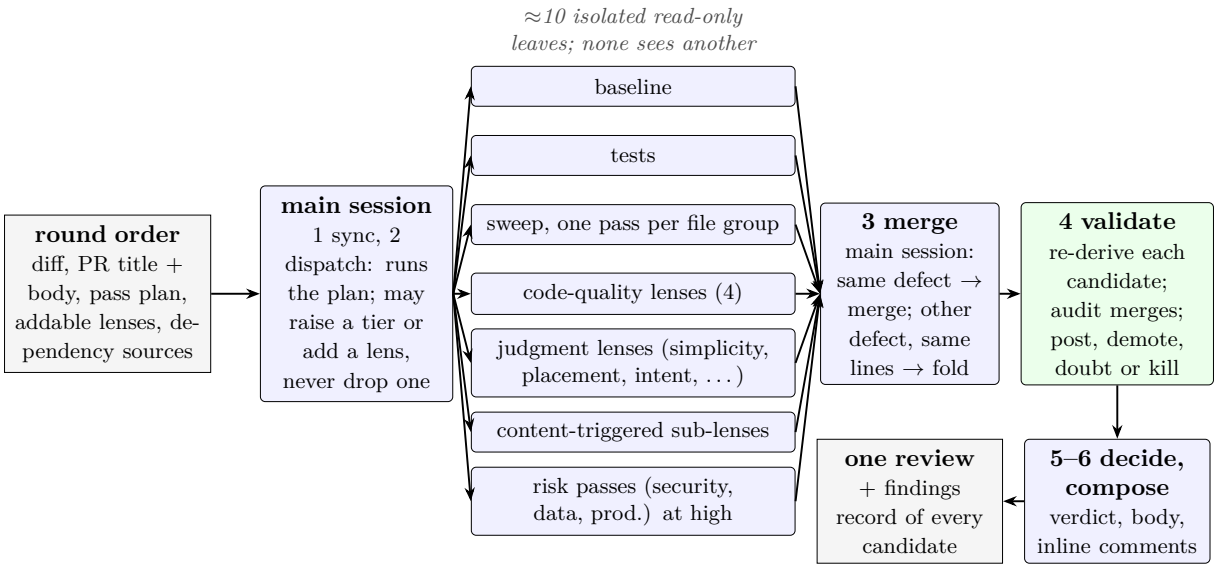


Figure 1: One QuickReview round, as the round template orders it. Steps 1–3 and 5–6 are the PR's main session; every pass and the validator are isolated agents. The validator runs last at high reasoning. On the gpt-6-sol default run, the passes took 75.5% of the cost, the validator 10.9% and the main session 13.5% (Section 6).

3 Design

This section describes the reviewer at the build every final number was measured on (`b2ce03a`); later commits change the reviewer's code, and a reader reproducing the results should use that build.

3.1 Principles

Independent angles. A review is the union of narrow passes, each a leaf agent that starts from the PR's diff and worktree, holds read-only tools, and does not see the other passes. Passes never launch other agents; the PR's main session is the only scheduler.

A check before posting. A separate validator re-derives every candidate from the code. Lenses are told that a plausible but wrong finding costs more than a missed nit, and the validator's job is to remove what does not survive contact with the code.

Read, never run. Every pass carries an execution policy: it never runs project code, tests, builds or package managers. Anything that needs execution is CI's job, and in production the reviewer reads CI results. To read what a library does, the round is given the library's source (below), not an environment that runs it.

Narrow procedures over broad instructions. Where a class of defect needs an exhaustive procedure — every consumer of a widened check, every file through a round trip — a dedicated sub-lens exists whose whole charter is to fill that table, so that a skipped row is visible rather than silent.

3.2 One round

QuickReview runs one main session per PR (Figure 1). The session receives a *round order*: the PR's merge base and head, its title and description, a *pass plan* computed by deterministic code from the changed files and the PR text, a list of *addable* lenses, and the location of any unpacked dependency sources. It then follows the round template's steps.

1. **Sync and dispatch.** The session launches every planned pass as a leaf agent, each with a task string naming its focus. It may *raise* a pass to a higher reasoning tier when the change's semantic complexity warrants it, and *add* an addable lens when it can name a concrete reason; it may never drop a planned pass or lower a tier. Raises and additions are recorded with their reasons.
2. **Merge.** The session collapses candidates that describe the same defect into one (*merge*, keeping every raising lens), and carries a *different* defect on the same changed lines as its own paragraph inside the surviving comment (*fold*). A P1 or P2 candidate is never folded.
3. **Validate.** The validator receives every candidate and the merge decisions, and gives each a disposition (Section 3.4).
4. **Decide and compose.** The session maps severities to posting tags, decides the verdict and writes the review: an opening paragraph, a section of points for a human to double-check, the verdict line, and at most 50 inline comments, each anchored inside a changed-line run of the diff (a finding with no valid anchor goes into the body).
5. **Record.** It writes a findings record of every candidate, the lens that raised it, its evidence and its disposition.

Severity and posting. Every finding carries a severity on a fixed scale (P0: certain production breakage, data loss or security hole; P1: a bug or contract violation that must not merge; P2: a probable defect needing an answer; P3: worth raising, not blocking; nit), a confidence, a claim and its evidence. The posting map is P0/P1 → `[blocker]`; P2 → `[blocker]` at confidence ≥ 0.8, otherwise `[question]`; P3 → `[question]` or `[nit]`. A reachability axis adjusts this: a finding whose trigger is contrived never blocks, and a P2 whose trigger is merely plausible posts as a question. Any surviving blocker makes the verdict REQUEST_CHANGES; otherwise the verdict is APPROVE. In `strict` nit mode (the default) the validator drops true-but-trivial candidates and a nit must name the change it asks for.

Behaviour lives in addenda, not prompt rewrites. Each lens is an agent file (a shared contract plus the lens's own instructions). Changes to a lens's behaviour are composed onto the built agent as named addenda from one registry (`PASS_ADDENDA`), scoped to all passes, to the lenses, or to named passes. At the measured build the registry holds about forty addenda, among them the execution policy, read batching, the reachability rules, the severity caps of the judgment and quality lenses, the validator's closed vocabulary of kill reasons and its demote, restate and doubt rules below, and a rule that a lens which finds one problem on some lines must look again for a second one.

3.3 The lens catalogue

Table 1 lists every lens at the measured build, what it looks for, and when the plan includes it. Selection is code, not a model decision: path patterns, content patterns in the added lines, sizes, and the length of the PR description. Three lenses are *risk passes* that always run at high reasoning; the others run at medium in the `default` preset (Section 3.5). The last column gives the share of the 185 PRs on which each lens ran in the gpt-6-sol default run.[2]

On the gpt-6-sol default run a round executed a mean of 11.2 passes including the validator (median 11, interquartile range 9–14, range 4–24); the plan averaged 10.9, and the session added passes on 36 of the 185 PRs. The `low` preset ran 11.0 on average with the same plans and added passes on 18 PRs (same source).

[2] `executed_passes` and `planned_lenses` in each case's `case-summary.json`, computed 2026-09-29 (`wp-data/passes.json`[3]). The gpt-6.1-sol run has the same plan on 181 of the 185 PRs: the presets and the model change tiers, not the plan.

Table 1: The lens catalogue at build `b2ce03a`. "Cap": the highest severity the validator lets the lens post. Last column: share of the 185 PRs on which the lens ran (gpt-6-sol default run). The validator runs last on every PR. [r] risk pass, always at high reasoning. [a] uncapped when it shows an outlier that breaks on a verifiable path. [b] may block only when it names the failure under the bad answer.

Lens	Looks for	Planned when	Cap	Ran
Always or nearly always				
baseline	claimed invariants, correctness, tests, conventions, sibling sites, duplicate fixes	every PR	—	100%
tests	existing tests, mocks and fixtures the change breaks; PR tests that mock the failure point away	a code file changes	—	100%
sweep	local slips read line by line: typos, fall-through, a copied line not updated, placeholders, unchecked parsing	one pass per group of ≤5 changed files and ≤500 lines, up to 6 groups; lockfiles, generated, vendored and prose files skipped	—	100%
Code quality (planned together)				
maintainability	stale docs and comments, misleading names, dead code, weak types, hidden coupling; never style	non-test code adds ≥20 lines	P2	77%
duplication	the same knowledge written twice, or again where the repo has it	same	P2	77%
abstraction	functions, types and contracts at the wrong level; leaky or speculative abstractions	same	P2	77%
consistency	alike things treated differently; a sibling left out	same	P2[a]	77%
style	breaks of the repository's own written style guide only	the repo writes a style guide	nit	10%
Judgment lenses				
simplicity	a simpler shape; checks defending states that cannot occur	≥150 changed code lines, deep nesting, or a file gaining ≥5 branches	P2	65%
intent	does the diff do what the title and body say	description has ≥60 words of author prose	question	45%
placement	the right home and abstraction level for new code	a new code file of ≥30 lines, or ≥8 code files	P2	30%
backcompat	breakage in the deploy window: old clients, in-flight messages, persisted state	API/serializer/handler or task code, schema files, migrations, feature flags	—	19%
Content-triggered sub-lenses (each narrows a broad lens)				
widened	every consumer of a changed predicate, guard or accepted-value list	a predicate or literal guard changes; ≥2 new IDs or ≥3 new cases	—	39%
claims	each checkable claim in the PR text, verified everywhere it applies	the PR text claims to fix, support or prevent something	—	32%
bounds	loops another party controls, size checks after allocation, unvalidated remote data, leaked host facts	matching patterns in added code	—	14%
rollout	old peer vs new code, new peer vs old code, old persisted state	the PR text mentions compatibility or rollout, or versioned state changes	—	10%
deferred	who wakes queued work, who reads before it drains, what coalescing merges	≥2 distinct deferral terms in added code	—	3%
roundtrip	sample files through read, decode, edit, encode, write	encoding handling and a write in added code	—	3%
Path-triggered domain lenses				
security[r]	injection, authorization, tenant isolation, secrets, cache safety	auth or permission paths, secret-shaped names, env files, added dependencies	—	14%
contracts	external payloads field by field, acceptance semantics	schema, serializer, integration or provider paths	—	11%
ui	frontend effects, mutations, async state	frontend file types	—	6%
delivery	at-least-once handlers, redelivery, drop paths	webhook, task, worker or consumer paths	—	4%
data-safety[r]	migration DDL, blast radius, backfills	migrations, SQL, backfills	—	3%
statelessness	in-process state under many replicas	module-level mutable state, cache or singleton files	—	2%
prod-risk[r]	behaviour changes to live endpoints, rollout, cascades	infrastructure, CI workflow, container files	—	1%
Addable only (the session adds them with a stated reason)				
concurrency	read-then-write races, walked step by step	never planned	—	5%
ask-human	questions whose answer lives outside the repository	never planned	question[b]	0%

Table 2: Validator dispositions at the measured build. "Posted" says whether the candidate reaches the review.

Disposition	When	Posted
post	confirmed as stated, at its audited severity	yes
demote	confirmed, but the consequence is smaller than claimed, or the lens's cap applies: lowered to P3, question or nit, never deleted for being small (*demote, don't kill*)	yes
restate	the mechanism is confirmed but the claimed consequence is wrong: restated with the true consequence at the severity that earns (*restate, don't kill*)	yes
merge	the same defect as another candidate, differently phrased	as one
fold	a different defect on the same changed lines, carried inside the surviving comment	inside it
doubt	the claim fails: `refuted` (counter-evidence read), `unverifiable` by reading, `bad-anchor`, or confidence below 0.5	no (kept in the record)
kill	a repeat of a settled point, out of scope, below the bar (a change that would not improve the code), or the wrong shape (a nit that asks for nothing, self-narration, style without a written guide)	no

Dependency sources. For the third-party packages a PR imports, the round downloads the packages' source archives at the versions the PR's lockfiles pin, from the npm and PyPI registries only, and unpacks them as plain text files; nothing is installed and no tool from an archive runs. The round order lists where they landed, so a pass can read and cite the library rather than its memory of it. A failure of any kind becomes a skipped entry; the round never waits on it.

3.4 The validator and its dispositions

The validator receives every candidate the lenses raised and the merge step's decisions. For each candidate it re-derives the claim by reading the code, checks it against points already settled in the PR's discussion, verifies the anchor, audits severity and confidence, and merges residual duplicates. Each candidate ends in one disposition (Table 2).

Doubt replaced deletion for validity vetoes between the gpt-6-sol runs and the measured build: a doubted candidate stays in the findings record, marked as possibly invalid, so that a validator at a lower tier cannot make a stronger lens's finding vanish without trace. For what reaches the review a doubt and a kill are the same, and the mechanism figures in Section 6, measured on the earlier build, count validity vetoes as kills.

3.5 Models, tiers and presets

Every pass runs at one of three tiers. The two cheaper tiers are chosen by diff size (below 150 changed lines, or above), the third is high; in the `default` preset both cheaper tiers map to medium reasoning, so the tier label records diff size rather than a reasoning level. The sweep and intent start at the cheapest tier, the three risk passes and the validator run at high, and the main session runs at the model's default (medium). The validator's tier is fixed and is never raised to match a lens.

One configuration word, the *preset*, sets the reasoning level and the lever plan (Table 3); an explicitly set key overrides it. The `default` and `low` presets share the *lean* plan, which was tuned for equal recall at lower cost (Appendix A); `max` enlarges it.

3.6 What the benchmark measures, and what it leaves out

The benchmark runs the production round, not a benchmark build: the same round template rendered by the production renderer, the same session persona, the same plan computation, the

Table 3: The three presets (`REVIEW_PRESETS` in `src/config.ts`). Every preset turns on the sub-lenses, the sweep, dependency sources, the tests lens, the code-quality lenses and the style lens.

	`low`	`default`	`max`
Reasoning	every role low	lenses medium; validator, risk passes high	every role at its ceiling
Sweep groups (max / files / lines)	6 / 5 / 500	6 / 5 / 500	8 / 3 / 400
Intent: words of author prose	60	60	20
Placement: code files / new-file lines	8 / 30	8 / 30	4 / 0

same built lens and validator agents with every addendum, and the same model configuration (with the base model replaced through configuration overrides). It differs in what cannot run offline, and those differences are stated to the session in a BENCHMARK MODE section appended to the round order:

- There is no GitHub, no CI and no memory. The CI gate on approvals is off; the session decides on the merits. Reviewer memory is off so that a foreign repository never reads or writes the production reviewer's memory.
- The session posts nothing. It writes four files instead: the inline comments exactly as it would post them, the review body, the findings record, and an outcome file with the verdict. The defect judge reads the comments and the body.
- The repository profile is the generic one, not the profile of the authors' production repository, whose framework-specific rules would be noise on these projects.
- The PR's title and description are given, as in production; dependency sources are fetched; one container runs each PR with a 45-minute limit.
- Production features outside a single first review are absent: rechecks after approval, replies to authors, escalation, the migration pre-check.

A switch that posts every true nit (`[review] nits = "open"`) exists because AACR-Bench's own metric rewards nits; it is off in every scored run.

4 Evaluation setup

The benchmark. Each of the 185 runnable PRs of AACR-Bench v1.0 [2] (ten languages) carries a list of verified defects — 520 in all: 90 high, 145 medium and 285 low severity. The key was built from an expert code hunt, the dataset's reference comments and seven reviewers' comments, each candidate checked against the code by agents that could not see who raised it, with 3,093 rejected candidates kept with their reasons [1]. A judge model (gpt-6-sol at high reasoning) reads each review whole and grades every defect found, partly found (counted as half) or missed, and every comment correct-valuable, correct-minor, partly correct, incorrect or unverifiable; "comments correct" is the share graded correct-valuable or correct-minor. Weighted recall weights high, medium and low severity 3, 2 and 1. A second judge from another model family agreed on 98.7% of verdicts in the earlier study. Every reviewer runs in a locked-down container that sees only the PR's base and head commits. The benchmark's own reference-comment metric is reported too (Section 5.4).

The contenders. Eight configurations ran once each on all 185 PRs: QuickReview's `default` and `low` presets and six third-party tools — pi-review [7], Claude Code [9], open-code-review [3], Codex CLI [4], OpenCode [5] and PR-Agent [8]. Each third-party tool ran in the best configuration found for it among the variants its own documentation ships or recommends, picked by a rule fixed in advance on 40 PRs not used to tune QuickReview [1]. Every OpenAI-based

Table 4: Runs behind this report. All on the same 185 PRs and the same key unless stated; all judged by the primary judge.

Run	Build, model	Used for
Final comparison	`b2ce03a`, gpt-6.1-sol (pi 0.99.1)	every outcome number in Sections 5–7
gpt-6-sol default run	`2d2036f`/`1ad710b`, gpt-6-sol (pi 0.87.1)	mechanism: pass counts, lens shares, cost split, per-lens attribution, validator dispositions (Section 6)
Selection set	as above, both models; 40 of the 145 PRs	the panel's internals (Table 10)
Base-model arms	`9106a64`, `b2ce03a`, `5cbd74a`	Section 8
Stability repeats	`9106a64`, gpt-6-sol, 8 PRs	Section 9

Table 5: The final comparison: defect benchmark, 185 PRs, 520 defects, primary judge. A partly found defect counts $\frac{1}{2}$. Weighted recall: high 3, medium 2, low 1. $ are list prices, not bills (gpt-6.1-sol at $2 / $0.10 / $10 per million uncached input / cached input / output tokens; Claude Code its own reported cost). Minutes are wall-clock under a shared scheduler and are not comparable across arms.[5]

Reviewer	Found	Recall	High (90)	Med. (145)	Low (285)	Weighted	Correct	$/PR	Median min/PR
QuickReview default	**409**	**79%**	**83.5**	**121**	**204.5**	**82%**	**95%**	1.36	9.5
QuickReview low	365.5	70%	79.5	104.5	181.5	74%	94%	0.88	6.7
pi-review panel (high)	347.5	67%	80	119	148.5	74%	89%	0.27	1.8
Claude Code (Opus 5.5)	324.5	62%	72.5	105	147	68%	81%	0.20	0.7
open-code-review high	298.5	57%	63.5	102	133	62%	86%	0.53	1.8
Codex CLI high	287	55%	81	101.5	104.5	65%	93%	0.14	2.1
OpenCode native + PR text	263.5	51%	76	99	88.5	61%	93%	0.17	2.3
PR-Agent	194	37%	55.5	70	68.5	44%	76%	0.07	1.9

reviewer ran on gpt-6.1-sol; Claude Code ran on Claude Opus 5.5 at its published medium `/code-review`. QuickReview itself was not re-tuned for this comparison.

Which run each number comes from. The report draws on several runs, and mechanism figures come from a different run than outcome figures (Table 4). Where a paragraph mixes them, it says which is which.

5 Results (RQ1)

Recall leads: high-severity defects first, then all severities; comment accuracy and cost follow. This section and the next three measure build `b2ce03a`, on which the design was studied; the shipping build is Section 10.

5.1 Defects found

The default finds 409 of the 520 defects (79%) with 95% of its comments graded correct (Table 5, Figure 2). In a paired bootstrap over PRs its lead over every other reviewer on overall recall excludes zero: +8.4 points [+5.6, +11.1] over `low`, +11.8 [+7.8, +15.7] over pi-review's panel, +16.2 [+10.8, +21.3] over Claude Code, and +21.3 to +41.3 over the rest (Table 6). Counting a partly found defect as missed changes no order (78.1% for the default, 66.0% for the panel).

Where the lead lies depends on whom it is over. Over the panel it is almost entirely low severity: high 83.5 against 80 and medium 121 against 119, but low 204.5 against 148.5. Over Claude Code, open-code-review, Codex CLI and OpenCode it includes medium severity (16–22

[5] `compare.py --cases all30+fresh10+rest145` on the runs listed in `bench/aacr/final/arms.txt`; reproduced independently from the judge files, 2026-10-01.

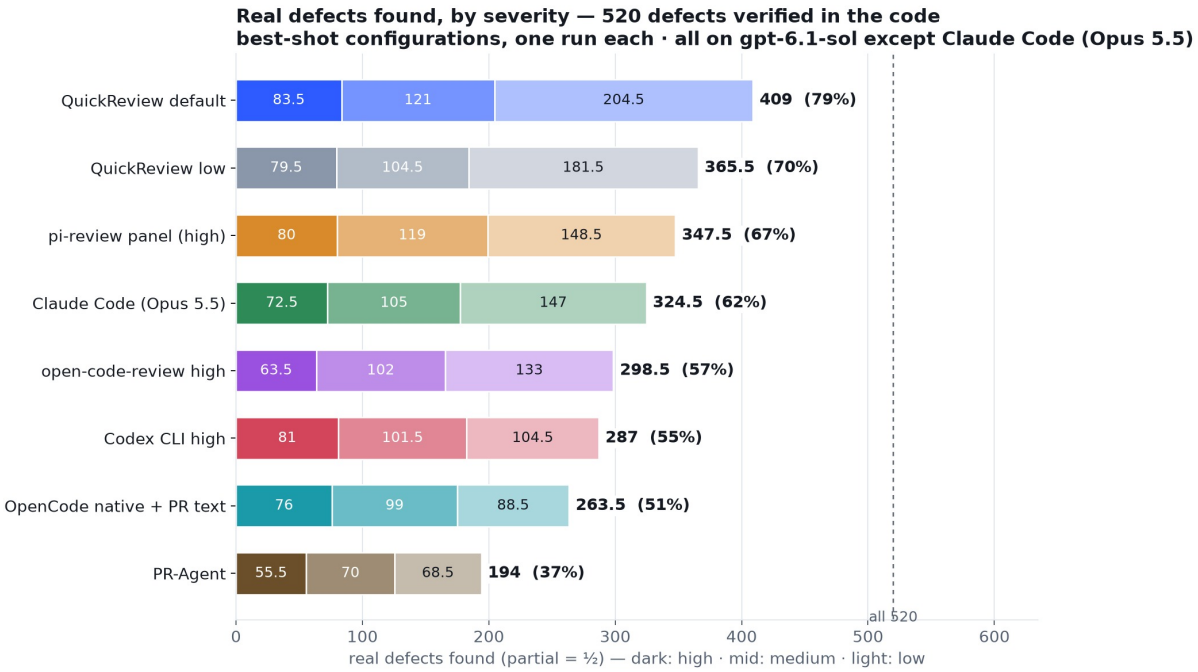


Figure 2: The final comparison: real defects found, split by severity (dark: high, mid: medium, light: low), 185 PRs, primary judge.

Table 6: Recall, high-severity recall and weighted recall with 95% bootstrap intervals over PRs (10,000 resamples, seed 20261001; PR sampling only, one run and one judge), and list cost per defect found (total over the 185 PRs divided by defects found).

Reviewer	Recall [95% CI]	High recall [95% CI]	Weighted [95% CI]	$/defect	$/high defect
QuickReview default	78.7% [74.8, 82.2]	92.8% [87.2, 97.4]	82.5% [79.1, 85.6]	0.61	3.01
QuickReview low	70.3% [66.2, 74.1]	88.3% [81.0, 94.7]	74.4% [70.5, 78.2]	0.44	2.04
pi-review panel (high)	66.8% [63.3, 70.4]	88.9% [82.8, 94.9]	74.1% [70.8, 77.4]	0.14	0.62
Claude Code (Opus 5.5)	62.4% [58.6, 66.4]	80.6% [71.1, 89.6]	68.0% [64.0, 72.1]	0.11	0.50
open-code-review high	57.4% [51.9, 62.5]	70.6% [58.8, 82.0]	62.4% [56.2, 68.1]	0.33	1.55
Codex CLI high	55.2% [51.0, 59.0]	90.0% [83.8, 95.5]	65.1% [61.4, 68.6]	0.09	0.33
OpenCode native + PR text	50.7% [47.1, 54.4]	84.4% [77.6, 90.8]	60.9% [57.3, 64.4]	0.12	0.42
PR-Agent	37.3% [33.7, 41.1]	61.7% [51.2, 72.5]	44.4% [40.1, 48.7]	0.06	0.22

more). On high and medium defects together the default finds 87.0% and the panel 84.7%, a lead of +2.3 points [−2.7, +7.0] that is not established [1]. On the 90 high-severity defects the default's lead over the panel (+3.9 points [−1.9, +9.6]) and over Codex CLI (+2.8 [−3.0, +8.7]) is not established; over `low`, Claude Code, OpenCode, open-code-review and PR-Agent it is. `low` is ahead of the panel by +3.5 points [−0.8, +7.7]: not established. Of the three defects that a read-only reviewer can find only by knowing the repository's CI, the default found 2, `low` 1, every other reviewer none.

5.2 What the comments are

The default posts the most comments, 5.4 per PR, and has the smallest share graded incorrect (2.5%) (Table 7). Its high share of correct comments rests partly on volume of small true things: a third of its comments are correct but minor, and only 56% map to a verified defect, against 81–87% for Claude Code, Codex CLI and OpenCode. In absolute terms it posts about 608

[7] Recomputed from the archived judge files, 2026-10-01 (scratchpad `paper-data/final-tables.md`, §1); unverifiable grades (≤0.7%) omitted.

Table 7: Every comment the judge graded, final comparison. Shares of each reviewer's comments. "Mapped": the judge tied the comment to a verified defect.[7]

Reviewer	Comments	Per PR	Correct-valuable	Correct-minor	Partly	Incorrect	Mapped
QuickReview default	999	5.40	60.9%	33.8%	2.6%	2.5%	55.6%
QuickReview low	801	4.33	59.4%	34.1%	2.4%	4.0%	58.2%
pi-review panel	585	3.16	72.6%	16.8%	4.6%	6.0%	67.0%
Claude Code	407	2.20	75.9%	4.7%	6.9%	11.8%	80.8%
open-code-review	570	3.08	69.5%	16.7%	4.7%	9.1%	63.3%
Codex CLI	343	1.85	91.0%	1.7%	2.9%	4.4%	85.4%
OpenCode	354	1.91	88.7%	4.0%	3.4%	4.0%	87.3%
PR-Agent	404	2.18	71.8%	4.0%	6.7%	17.6%	76.5%

correct-valuable comments to the panel's 425. Whether the extra minor comments are worth a reader's time is not something this benchmark measures (Section 11).

5.3 Where the lead comes from

By kind of defect. The largest margin is on maintainability defects (72 defects): the default finds 81% against 16–51% for the third-party tools, which fits the code-quality lenses. Those lenses were designed on the tuning PRs, so this is also where an alignment between the design and the key's notion of a defect would show first. Without the maintainability defects the default finds 78% against 71% for the panel. On incomplete fixes (57 defects) it finds 89% against 73%. On the largest class, functional bugs (229 defects), the margin is small: 77% against 73% for the panel and 68–69% for Claude Code and open-code-review. [8]

By PR size. The lead grows with the size of the change. On PRs of at most 50 changed lines the default finds 70%, against 67% for Claude Code and 62% for the panel; on 51–200 lines 76% against 69%; on 201–1,000 lines 82% against 67% for the panel and 60% for Claude Code. A reviewer that reads every changed file in small groups should gain most where a single read has most to skim past; the per-file sweep is the candidate mechanism (Section 6).

Tuned and unseen PRs. QuickReview's levers were designed from its misses on the original 40 PRs; the other 145 were never looked at before its final build. It scores lower on the 40 (70%) than on the 145 (81%), and so does every other reviewer, so the 40 are harder for everyone. Its margin over the mean of the other seven reviewers is 18.3 points on the 40 and 22.5 on the 145, the opposite of what tuning on the 40 would produce. On the 145 PRs not used to choose the competitors' configurations it leads the panel 77% to 66%.

Without reviewer-sourced defects. On the 369 defects the key would hold if no reviewer's comment had been a candidate (those raised by the code hunt or a dataset reference), the default finds 81%, `low` 74%, Claude Code 70% and the panel 69%. On a key that drops only the 28 defects (24 low) whose sole sources are QuickReview's own earlier runs, the default finds 79.4% of 492, `low` 71.4% and the panel 68.7%; the lead over the panel is +10.7 points [+6.5, +14.7] and no order changes [1].

5.4 AACR-Bench's own metric

On the benchmark's published metric — how closely a review reproduces the dataset's 1,418 reference comments on these PRs, at least 80% of them AI-written — the default has the highest recall (10.7%) and F1 (12.5%); open-code-review is next on F1 (11.6%), and PR-Agent (21.3%)

[8] Scratchpad `paper-data/final-tables.md`, §3b and §4.



Figure 3: Cost per PR against defects found, final comparison. QuickReview's two presets sit alone at the top right; the six other tools lie between $0.07 and $0.53 per PR.

and open-code-review (20.2%) have the highest precision against the default's 15.2%. With the 80 references that are rewrites of earlier review bots' comments removed, the default and open-code-review stay first and second on F1 (11.4% and 10.8%). The companion paper [1] explains why every number on that metric is low and why its order differs from the defect benchmark's.

6 Cost: where it goes and what it buys (RQ2)

QuickReview's default spends $1.36 per PR at list prices: 2.6 times open-code-review, 5 times the panel, 7–9 times Claude Code, OpenCode and Codex CLI, 21 times PR-Agent; it reads 1.88M input tokens per PR (75% of them cached) and writes 27k, 5.1–24 times the output of the third-party tools (Figure 3). Unless stated, the mechanism figures in this section are from the gpt-6-sol default run (Table 4), on which the default cost $1.71 per PR and found 437.5 defects; the gpt-6.1-sol run's split is unmeasured.

6.1 Where the money goes

The lenses take 75.5% of the cost, the validator 10.9% and the main session 13.5%. Within the lenses the spend follows how often a lens runs and how much it reads (Table 8): the sweep alone is 13.6% of the total, the baseline 9.4%, the tests lens 7.1%, and the four code-quality lenses together 22.1%.

Redundancy is large and deliberate. Of the 440 defects credited to the default on the gpt-6-sol run,[9] only 71 were raised by a single lens and 126 by six or more (two lenses 57, three 72, four 60, five 54). The validator saw 2,959 candidates: 1,456 merged as duplicates of another finding — the redundancy, collapsed — 1,034 posted, 164 demoted and posted, 207 killed (7.0%), 64 withheld and 34 folded.

Its value is uneven. A lens's *unique* finds — defects for which every crediting comment names that lens alone — are the best offline proxy for what dropping it would lose (Table 8). Two lenses carry most of them: the sweep (29 unique finds, 2 of them high, for 13.6% of the cost) and maintainability (15, mostly stale documentation, for 5.4%). At the other end, abstraction and duplication together cost 9.9% of the budget for one unique find in 185 PRs; placement, bounds, contracts and style found nothing no other lens found. Shared credit is large for every lens — consistency co-found 237 defects — which is the redundancy above: a lens with no unique

[9] The attribution counts a partly found defect once; Table 12 counts it as half.

Table 8: Per-lens cost and credit, gpt-6-sol default run, 185 PRs. Cost share: of the run's total. Co-credit: defects at least one of whose crediting comments names the lens. Unique: defects whose crediting comments all name that lens alone. Rows naming several lenses add the lenses' figures, so a defect two of them co-credit counts twice.

Lens	Cost share	Co-credit	High	Unique
sweep	13.6%	334	74	29 (2 high)
validator	10.9%	—	—	—
baseline	9.4%	285.5	72	5
tests	7.1%	79	31	3.5
consistency	6.8%	237	60	2
maintainability	5.4%	125.5	26	15
abstraction	5.0%	100	26	0
duplication	4.9%	71	14	1
simplicity	4.1%	59	15	3
widened	3.3%	124.5	30	1
intent	2.9%	66	14	2
claims	2.6%	90	21	1
placement	1.9%	29	6	0
security	1.3%	28.5	7	1
backcompat	1.3%	18	6	3
bounds, contracts	2.0%	57	20	0
seven smaller sub-lenses	2.7%	80.5	—	4
data-safety, prod-risk, style	1.1%	14	5	0
main session	13.5%	—	—	—

finds may still make a find more likely in a run where the lens that usually raises it misses. [10] Dropping the sweep is estimated to cost about 5.6 points of recall.

The validator removes little that is real. For every defect the default did not fully credit, gpt-6.1-sol labelled each finding its validator killed or withheld within five lines of the defect as stating the defect, related to it, or about something else. It read the defect's statement, counting condition and location and the finding's text alone (no code), and was not told which items were posted or by which arm; the labels are one model's and were not checked by hand. Of the 520 real defects, it labelled a dropped finding as stating 5 that the default missed outright — 1.0 point of recall. The default's misses are recognition misses, not filtering: of the 80 defects it missed outright (82.5 missed credits, counting a partial find as half), 56 had a comment posted within five lines that the judge credited to something else, 11 had a candidate elsewhere in the file, 7 had nothing raised in the file, and 6 had nothing posted near them but a finding at the site that was not posted. These classes go by position, not by label, and a dropped finding can also sit beside a posted comment, so they do not bound what posting the drops would recover. Posting every killed or withheld finding would add 23% more comments for at most 2.7 points if the labels are right (1.0 counting only the findings labelled as stating the defect; 5.3 if every dropped finding within five lines counted).

6.2 What the compute buys

With the design fixed, less compute finds less. The `low` preset — the same plan, every role at low reasoning — finds 70% at $0.88 per PR against 79% at $1.36; the difference is established. On the gpt-6-sol runs, where the gap was larger (84% against 66%), an attribution of the lost credit put 55% of it in lens generation (no posted comment and no killed or withheld

[10] Per-lens offline analysis of the gpt-6-sol default run's findings records, 2026-09-29 (scratchpad `tuning-writeup.md`, Q1–Q3). "Unique" is fragile: the other lenses' overlap varies from run to run.

finding within five lines of the defect was about it), 20% in validator filtering (only a finding low's validator dropped was about it; 22 of the lost defects), and 26% in weaker comments that were about the defect but earned less credit (shares rounded). The shares are of the gross lost credit, the 109.5 credits on 116 defects that the default earned and low did not. Of the defects each arm left with no credit, 32 at low had every finding about them dropped by the validator, against 7 in the default, whose validator runs at high. Whether a comment or a dropped finding near a defect is about it is gpt-6.1-sol's label, as above: stating the defect or related to it, from the text alone, without being told which items were posted. A smaller base model that writes about half as much per pass collapses to 28% (Section 8): what the lenses do not raise, the validator and the session cannot recover.

Marginal cost. Average cost per defect favours cheap reviewers by construction; the figure that matters for choosing a configuration is the cost of the defects one buys by moving up. Over the 185 PRs, moving from `low` to the default costs about $89 more for 43.5 more defects (about $2.05 each, 4 of them high severity); moving from the panel to the default costs about $201 more for 61.5 more defects (about $3.27 each), 56 of them low severity.[11]

The counterpoint. QuickReview's case is recall, not cost efficiency. Every other tool is cheaper per defect found ($0.06–$0.33, against $0.61 for the default and $0.44 for `low`; Table 6). On high-severity defects alone the panel and Codex CLI are not separated from the default, and they find a high-severity defect for $0.62 and $0.33 against $3.01. A team that wants the most important findings cheaply is better served by one of those tools at its best configuration; QuickReview is the choice when a missed medium- or low-severity defect costs more than the $2.05–$3.27 of review that each extra defect found costs above. All costs are list prices; the OpenAI-based arms ran through a subscription login, so they are not bills. Ratios and differences of costs are computed from the unrounded per-PR costs, so they can differ by one unit from a ratio of the rounded figures printed beside them.

7 The closest competitor: pi-review's panel (RQ3)

Of every third-party configuration we ran, pi-review's three-reviewer panel comes closest to QuickReview, and its design is also the closest. This section compares the two designs, the results on all 185 PRs, and what the comparison can and cannot say about design versus budget.

7.1 Two designs

The panel is `pi-review --mode code --panel code-experts`, which is what pi-review's `/rv` command runs by default in the pi host [7, 6]. The `code-experts` preset defines three reviewers, for correctness, security and testing; each is an isolated, read-only pi process with its own role prompt, and all three review the same diff without seeing each other. Their findings are clustered — by a deterministic match on path and summary, then, for ambiguous pairs on the same path, by a model adjudicator that may merge findings but not invent, drop or rewrite them — and a cluster raised by at least two reviewers is *confirmed*; a singleton stays visible as an *advisory*. Table 9 sets the two designs side by side.

7.2 Results on all 185 PRs

The panel finds 347.5 of 520 defects (67%) against the default's 409 (79%), 80 of the 90 high-severity defects against 83.5, at a fifth of the cost. Of the 61.5 defects between them, 56 are low

[11] Computed from the whole-benchmark list costs ($251, $162, $50; scratchpad `paper-data/final-tables.md`, §5) and Table 5.

Table 9: The two multi-angle designs as they ran in the final comparison.

	QuickReview default	pi-review panel
Angles	≈10 lenses planned from the change, plus a sweep over every changed file	3 fixed specialists
Independence	isolated leaves; none sees another	isolated reviewers; none sees another
Deciding step	a validator re-derives each candidate from the code	a vote: confirmed if ≥2 of 3 agree; singletons kept as advisories
What a single angle's real find becomes	posted, demoted if in doubt	an advisory; shown, but by pi-review's rule non-blocking
Inputs	diff, worktree, PR title and body, dependency sources	diff, worktree (read-only tools)
Reasoning	lenses medium; validator and risk passes high	every reviewer high
Tokens per PR	1.88M input (474k uncached), 27.0k output	0.35M input (89k uncached), 4.7k output
$ per PR	1.36	0.27

severity, 2 medium and 3.5 high. The default's lead is established (+11.8 [+7.8, +15.7]); `low`'s 3.5 points over the panel, at 3.2 times its cost, are not.

What the vote does. We score confirmed and advisory findings alike, because both are in the panel's output; this is the generous reading. Re-reading the same judgements with confirmed clusters only, the panel finds 250 of 520 defects (48%) and 69 of 90 high-severity ones; the singletons carry 97.5 of its 347.5 credited defects. The vote does not separate right comments from wrong ones: 90% of confirmed comments are correct (264 of 292) against 88% of advisories (259 of 293). The confirmed-only figures re-read the judgements rather than re-judge; a panel that posted only confirmed findings could be graded slightly differently.[12] So the panel's recall comes from its independent angles, including the angles that only one reviewer took, and not from its agreement check.

The model behind both. On the 40-PR selection set, where we also read the panel's clustering output, both systems ran on gpt-6-sol and gpt-6.1-sol (Table 10). The panel's comment accuracy rose from 78% to 91% with the newer model; its recall moved by two defects. One pi-review reviewer at high had 71% of its comments correct and 62% recall on gpt-6-sol, the panel 78% and 68% — at 2.8 times the cost.

Table 10: QuickReview and pi-review's panel on the 40 selection PRs (115 defects: 17 high, 33 medium, 65 low), primary judge, one run each. "Confirmed only" re-reads the same judgements, counting only comments from clusters that two or more reviewers raised. Tokens: per-PR mean, uncached and cached input, then output, summed over every call.

Reviewer	Reading	Found	Recall	High	Medium	Low	Correct	$/PR	Input (uncached / cached)	Output
QuickReview default, gpt-6.1-sol	all comments	98	85%	17	26	55	94%	1.64	569k / 1,793k	32.0k
QuickReview low, gpt-6.1-sol	all comments	86.5	75%	17	23.5	46	93%	1.04	384k / 951k	17.9k
pi-review panel, gpt-6.1-sol, high	confirmed + advisory	80	70%	17	27	36	91% (139)	0.30	101k / 370k	5.8k
	confirmed only	50	43%	11	17	22	90% (61)			
pi-review panel, gpt-6-sol, high	confirmed + advisory	78	68%	15	25.5	37.5	78% (162)	0.46	120k / 702k	8.5k
	confirmed only	46	40%	13	17	16	79% (63)			

[12] FULL-BENCHMARK.md, "Why pi-review's panel is the strongest competitor".

7.3 Why QuickReview finds more: candidate explanations

Four differences could account for the low-severity gap. None was ablated on the unseen PRs, so these are hypotheses with partial support, not findings.

- **The per-file sweep.** The panel has no pass that walks every changed file; each specialist decides where to look. On the gpt-6-sol run the sweep was the only route to 29 credited defects, 22 of them low, and dropping it is estimated to cost about 5.6 points. The lead growing with PR size is consistent with this.
- **More passes and more reading.** About ten lenses to three reviewers, 5.3 times the input and 5.7 times the output.
- **A validator that demotes.** A real find by one QuickReview lens is posted, demoted if in doubt; a real find by one pi-review specialist stays an advisory. Under our scoring both count, so this explains a difference in what reaches a developer's attention more than a difference in recall.
- **More context.** QuickReview reads the PR text and dependency sources. For pi-review the PR text was a small lever: one reviewer at high gained 1.5 defects from it on the 40 PRs, and the variant lost its selection.

The comparison is asymmetric in both directions: QuickReview had more context and five times the budget; the panel's reviewers ran at high reasoning where QuickReview's lenses ran at medium.

7.4 Design or budget?

The strongest competitor is the design closest to QuickReview's, several independent angles followed by a decision over what they raised. It is tempting to read that as evidence that the approach, and not only the budget, carries the result. The data support a narrower statement.

What the data show. Independent angles help pi-review: three reviewers find more than one, and the extra recall comes from findings only one reviewer raised. Within QuickReview, budget and model matter a great deal: `low` loses 8.4 points, a small model 56 (against gpt-6-sol). Spending more does not by itself buy recall in every tool — OpenCode through our bridge cost $0.96 per PR on gpt-6-sol and found 43% on the 40, open-code-review at high cost $1.12 and found 68%, and the panel matched the latter at $0.46 — but those are differences between tools, not between budgets for one design.

What they do not show. Every comparison that adds angles also adds budget: the panel costs 2.8 times one reviewer, QuickReview five times the panel. We ran no equal-budget alternative — a panel with as many reviewers as QuickReview has lenses, a single reviewer at maximum effort, Claude Code at its broader effort levels — and no lens ablation on the unseen PRs. Whether QuickReview's design beats a different way of spending $1.36 per PR is open. **[NEED EXPERIMENT: an equal-budget arm, e.g. a 9–10-reviewer pi-review panel or a single high-effort reviewer at about $1.36/PR, on the 145 unseen PRs]**

8 Base model (RQ4)

This section changes only the model behind every role of QuickReview — the main session, the lenses, the validator and the risk passes — and holds the design, the `default` preset and the reasoning levels fixed. Every arm was judged by the primary judge against the same key (Tables 11–14). The gpt-6.1-sol row is the final comparison's.

[14] Computed from runs `v2-default-*`, the original default runs, `g61-smoke-smoke10` and `g61-{dev10,holdout10,fresh10,rest145}`, and `all-luna-*` with `compare.py`, 2026-09-29/30.

Table 11: Base-model arms. Each run's `run.json` records the `--config` overrides that set every role's model, each role keeping its reasoning level.

Arm	Model	pi	Build
gpt-6-sol, current build	`openai-codex/gpt-6-sol`	0.87.1	`9106a64` (re-run 2026-09-29)
gpt-6-sol, original run	`openai-codex/gpt-6-sol`	0.87.1	`2d2036f` / `1ad710b`
gpt-6.1-sol	`openai-codex/gpt-6.1-sol`	0.99.1	`b2ce03a`
gpt-6-luna	`openai-codex/gpt-6-luna`	0.87.1	`5cbd74a`
Claude Opus 5.5	`anthropic/claude-opus-5-5`	0.87.1	`5cbd74a` (40 PRs only)

Table 12: QuickReview default on four base models, 185 PRs, 520 defects, primary judge. AACR columns: harness `all` variant, 3 rounds, ±1 line gate; comment count in brackets. Every row is 185/185 judged.[14]

Base model	Found	Recall	High (90)	Med. (145)	Low (285)	Weighted	Correct	$/PR	Median min	AACR R	AACR P (comments)	AACR F1
gpt-6-sol, current build	436	84%	85	121.5	229.5	86%	88%	1.71	8.8	11.7%	13.5% (1,227)	12.6%
gpt-6-sol, original run	437.5	84%	86	124.5	227	87%	86%	1.71	8.9	11.6%	13.8% (1,191)	12.6%
gpt-6.1-sol	409	79%	83.5	121	204.5	82%	95%	1.36	9.5	10.7%	15.2% (999)	12.5%
gpt-6-luna	147	28%	39	44	64	32%	79%	0.04	4.5	4.7%	22.1% (300)	7.7%

Table 13: The original 40 PRs (126 defects; QuickReview's tuning PRs), primary judge.[15]

Base model	Found	Recall	High (34)	Med. (31)	Low (61)	Weighted	Correct	$/PR	Median min	AACR R / P / F1
gpt-6-sol, current build	100.5	80%	31.5	26	43	84%	89%	1.54	8.4	10.2% / 14.3% / 11.9%
gpt-6-sol, original run	102	81%	32	26	44	85%	89%	1.53	8.8	10.7% / 15.6% / 12.7%
gpt-6.1-sol	88	70%	29.5	25	33.5	76%	95%	1.18	9.5	10.8% / 17.5% / 13.4%
gpt-6-luna	31.5	25%	11.5	10	10	29%	77%	0.04	4.4	3.8% / 18.8% / 6.3%
Claude Opus 5.5	**119.5**	**95%**	**32**	**30.5**	**57**	**95%**	**94%**	3.24	8.4	11.3% / 13.1% / 12.1%

Table 14: The same 40 PRs split into the 30 tuning PRs and fresh10.

Base model	30 tuning PRs: found / 97	fresh10: found / 29
gpt-6-sol, current build	74.5 (77%)	26 (90%)
gpt-6-sol, original run	74.5 (77%)	27.5 (95%)
gpt-6.1-sol	66.5 (69%)	21.5 (74%)
gpt-6-luna	24.5 (25%)	7 (24%)
Claude Opus 5.5	91.5 (94%)	28 (97%)

gpt-6.1-sol trades recall for accuracy and cost. It finds 27 fewer defects than the current gpt-6-sol build (409 against 436, 5 points), 25 of them low severity; high is 83.5 against 85 and medium 121 against 121.5. In exchange 95% of its comments are correct (against 88%) and it costs 21% less ($1.36 against $1.71). Its AACR F1 is the same (12.5% against 12.6%): it posts fewer comments (999 against 1,227) at higher precision. The gap is larger on the original 40 (70% against 80%) than on the new 145 (81% against 85%). The final comparison therefore shows QuickReview on the model that is not its best measured OpenAI configuration; on gpt-6-sol the default found 84%. On smoke10 alone gpt-6.1-sol looked level with gpt-6-sol (26 found against 24.5 and 25): a 10-PR smoke run can find a broken arm, not rank two working ones.

The pi version is a confound we bounded only for cost. gpt-6.1-sol needs pi 0.99.1; every other arm ran on pi 0.87.1. One check of gpt-6-sol on pi 0.99.1 (`infiniflow__ragflow@7051234`) cost the same as the original run on that PR ($1.52) and found 3 of 3 defects against 2 of 3: it rules out a large cost change from the pi version, not a small recall effect.

[15]Computed from runs `opus-{smoke10,dev10,holdout10,fresh10}` and the runs of Table 12 cut to the same 40 PRs, with `compare.py`, 2026-09-29/30.

gpt-6-luna collapses. It finds 28% of the defects and 39 of the 90 high-severity ones. The same passes run (11.1 subagent passes per PR on rest145, against 11.4 for gpt-6-sol), but each writes about half as much (median 963 output tokens per pass, against 2,194), and the reviews post a quarter as many comments (300 against 1,227). What the lenses do not raise, the rest of the design cannot recover. Its $0.04 per PR is pi's list price for luna.

Claude Opus 5.5 finds more, at twice the cost. On the 40 it finds 119.5 of 126 against 102 for the original gpt-6-sol run; high severity is level (32 each), and the gain is 4.5 medium and 13 low. It costs 2.1 times as much ($3.24 against $1.53) in the same wall-clock time, and its AACR F1 is no higher. Almost all of the gain is on the 30 tuning PRs (91.5 against 74.5); on fresh10 both are near the ceiling. This is 40 in-sample PRs and one run, not a 185-PR result.

The same model in two harnesses. Claude Code runs on the same Claude Opus 5.5. On the same 40 PRs it found 75 of 126 (60%) against 119.5 for QuickReview on Opus, at about a twentieth of the cost ($0.16 against $3.24 per PR). This is a contrast, not a measured harness effect: the two differ in budget, in inputs (QuickReview reads the PR description), in effort (Claude Code ran its medium, most-confident-findings setting) and in design at once, and the 40 are QuickReview's tuning PRs.

`max`: an anecdote. The `max` preset (every role at gpt-6.1-sol's ceiling, the full lever plan) ran once, on smoke10 only. Three of the ten PRs timed out at the 45-minute limit after one retry. On the seven it completed it found 13 of 16 defects against 10 at the default, at 3.7 times the cost ($3.08 against $0.84) and 3.9 times the median time (27.0 against 7.0 minutes); counting the timed-out PRs as empty reviews, it found 13 of 34 against the default's 26.[16]

Reading. The design does not make the model irrelevant: with the design fixed, recall ranges from 28% to 84% on the 185 PRs, and a weaker model's misses start at the lenses. Costs in this section are pi's list prices per call, not bills; the Opus arm ran through the Anthropic API and is priced at Anthropic's list price.

9 Stability (RQ5)

A full reproduction. The gpt-6-sol default was re-run on all 185 PRs on 2026-09-29 on a later build (`9106a64`), after three reviewer changes: the validator's validity veto became a doubt rather than a deletion, three passes were made stack-neutral (their framework-specific rules moved to repository-profile addenda), and the style lens learned to read style guidance from agent-directive files. The re-run found 436 defects against 437.5, 85 high-severity against 86, and scored AACR F1 12.6% against 12.6%; comments correct moved from 86% to 88%. On the unseen 145 the two runs find the same number (335.5); the whole 1.5-defect difference is on the original 40. This reproduces the gpt-6-sol result; the final gpt-6.1-sol run was not repeated.

Run-to-run spread. The gpt-6-sol default ran four more times (`stab-r1` … `stab-r4`, build `9106a64`) on 8 PRs. Two of the eight have no defect in the key, so recall rests on six PRs and 17 defects (Table 15).[17] The six runs range from 11 to 14 found, recall 65–82%; most of the spread is one PR (`wavetermdev__waveterm@61b0fb4`, 2 to 4 of its 5 defects), and the one high-severity defect was found every time. On 17 defects a gap of 3 is within noise, and the sample is too small for a variance estimate. Earlier, on the 30 tuning PRs, two runs of one build differed by 2.5 of 97 defects (RESULTS.md, "Noise"). No other reviewer was repeated.

[16] Run `g61max-smoke-smoke10` against the smoke10 runs of the other arms, `compare.py`, 2026-09-29/30.

[17] Runs `stab-r{1..4}-{dev10,fresh10,rest145}`, the `v2-default-*` runs and the original default runs, cut to the same 8 PRs, `compare.py`, 2026-09-29/30.

Table 15: Six runs of QuickReview default (gpt-6-sol) on the same 8 PRs (17 defects on the 6 PRs that have any), primary judge.

Run	Found / 17	Recall	High (1)	Med. (5)	Low (11)	Correct	$/PR
stab-r1	11	65%	1	3	7	76%	1.97
stab-r2	14	82%	1	5	8	70%	1.92
stab-r3	12.5	74%	1	4	7.5	78%	2.06
stab-r4	11.5	68%	1	5	5.5	73%	2.03
current build, full re-run	13.5	79%	1	5	7.5	79%	2.01
original run	13.5	79%	1	5	7.5	64%	1.94

10 The frozen build

Everything above measures build `b2ce03a` (2026-10-01). Over the next three days the reviewer was changed once more, from the misses of that build on the stored benchmark runs, and frozen as build `e32f0cf` (2026-10-04), still on gpt-6.1-sol. The changes:

- *Two lenses.* A behaviour-delta lens compares base and head on every modified or deleted line of non-test code, planned only where base behaviour is at risk (a removed or replaced guard, condition, call, return or default, a reordered statement, existing code wrapped in a new branch, a deleted file). A lint-conformance lens is planned only when a CI job runs a linter or formatter over a changed file.
- *Shared instructions.* Once a mechanism is confirmed, check its sibling sites, its twin, every consequence and the other failures in the same loop; anchor a finding at its cause, not its symptom; report one-token slips on changed lines (an operator, a bound, a loop target, a near-namesake); and no token, time or finding budget applies. The style lens walks every rule the repository writes down.
- *Coverage.* After the lenses, changed non-test source files on which no lens raised anything get a targeted sweep; a wide change's sweep is packed by a line budget rather than capped by file count.
- *One comment per root cause.* A deterministic merge clusters findings that share a site, a named identifier or a fix target, and only across different validator groups, so it never overrules the validator.
- *Effort.* The baseline, the sweep and the behaviour-delta lens run at high reasoning effort.
- *Prompt cleanup.* Contradicting rules resolved in favour of the newer, shared addenda merged and compressed, so the composed lens instructions are shorter. No lens was removed.

Results. The frozen build finds 87.5 of the 90 high-severity defects (97.2%), ahead of the panel by +8.3 points [+1.9, +14.6] and of Codex CLI by +7.2 [+0.6, +14.1]; 94.5% of the high- and medium-severity ones, ahead of the panel by +9.8 [+5.1, +14.3], where `b2ce03a` was level with it; and 461 of all 520 (88.7%, 95% CI 85.7–91.4), 10.0 points more than `b2ce03a` [+6.8, +13.3] and 21.8 more than the panel [+17.8, +25.7] (Table 16).[18] The gain over `b2ce03a` is similar on the PRs whose misses were mined (9 points), on those used for validation (14) and on the 20 not used (13).

[18]Paired bootstrap over PRs, 10,000 resamples, seed 20261001, recall as a ratio of sums, from the primary judge's verdict files; the companion paper's Section "The frozen build" carries the full comparison [1].

Table 16: The frozen build, build `b2ce03a` and the three strongest third-party tools on the same 185 PRs and 520 defects, primary judge, same key, one run each; recall first, then what the comments are, then cost and time. R145: misses mined; smoke10 and dev10: used to validate the changes; holdout10 and fresh10: not used for them. List prices, not bills; minutes are wall-clock under a shared scheduler.

	frozen `e32f0cf`	`b2ce03a`	pi-review panel	Claude Code	Codex CLI
High-severity recall (90) [95% CI]	**97.2%** [93.9, 99.5]	92.8%	88.9%	80.6%	90.0%
High + medium recall (235) [95% CI]	**94.5%** [91.2, 97.2]	87.0%	84.7%	75.5%	77.7%
Found (520)	**461 (89%)**	409 (79%)	347.5 (67%)	324.5 (62%)	287 (55%)
high / medium / low	87.5 / 134.5 / 239	83.5 / 121 / 204.5	80 / 119 / 148.5	72.5 / 105 / 147	81 / 101.5 / 104.5
R145 (394)	356 (90%)	321 (81%)	270 (69%)	249.5 (63%)	226 (57%)
smoke10 + dev10 (71)	59.5 (84%)	49.5 (70%)	38 (54%)	39 (55%)	32 (45%)
holdout10 + fresh10 (55)	45.5 (83%)	38.5 (70%)	39.5 (72%)	36 (65%)	29 (53%)
Comments per PR	6.9	5.4	3.2	2.2	1.9
valuable / minor / incorrect	51 / 42 / 4.5%	61 / 34 / 2.5%	73 / 17 / 6.0%	76 / 5 / 11.8%	91 / 2 / 4.4%
$/PR (mean / median)	2.06 / 1.87	1.36 / 1.19	0.27	0.20	0.14
Median min/PR	17.5	9.5	1.8	0.7	2.1

What it costs. $2.06 per PR on average (median $1.87), 51% more than `b2ce03a` and 4 to 31 times the third-party tools; a median 17.5 minutes per PR, 1.8 times `b2ce03a`'s 9.5, against 0.7–2.3 minutes for the third-party tools. It also says more: 6.9 comments per PR, 42% of them true but minor, and 4.5% graded incorrect against 2.5%.

External checks. On Martian's 50 PRs, scored by one gpt-6.1-sol judge with three tools' published comments, the frozen build finds 75 of the 86 golden comments an audit judged real (87%), against 68 for qodo-extended-v2 (level with it on the `test30` half, 43 of 52 each), and has the highest Martian recall (65.8) and the lowest precision (16.2). On Qodo's PR-Review-Bench, with all thirteen tools judged by our judge, it has the highest recall of functional bugs (52.1%) and the fourth-highest F1 (40.9); its recall of rule violations rose from 14.8% to 37.6%, and on the 31 Qodo PRs its development did not use, its recall rose by 7.8 points over the 2026-10-02 build (95% CI +2.1 to +14.1). Both benchmarks count a real defect missing from their lists as a false positive [1].

The same three third-party harnesses as in the main comparison (open-code-review, PR-Agent and OpenCode, each in its configuration there on gpt-6.1-sol) also ran on both benchmarks, which makes those rows a comparison of harnesses on one model. On Martian's audited real goldens they find 67, 46 and 48 of 86 against the frozen build's 75; on Qodo's they find 27–29% of all issues against 45.3%, 44–46% of the functional bugs against 52.1%, and 8–12% of the rule violations against 37.6%. They win on precision on both benchmarks, and on F1 on Martian only: there all three have a higher F1 than the frozen build's 26.1 (33.1–38.8), and open-code-review a higher F_2 (47.4 against 40.9) and full coverage of the real Critical and High goldens on one more PR (16 of 19 against 15). On Qodo the frozen build keeps the highest F1 of the four, 40.9, with OpenCode's 40.0 the nearest. They cost $0.09–0.86 and take 1.2–3.3 minutes per PR on these benchmarks, against the frozen build's $2.84 and 26.1 minutes on Martian and $2.48 and 19.6 on Qodo.

11 What these numbers do not show

The benchmark's authors built this reviewer. The answer key was built partly from candidate defects in QuickReview's own earlier reviews (on gpt-6-sol it raised 88% of the defects in the first build of the key for the 145 new PRs, 29 of them raised by no other source), by agents orchestrated by the same assistant that designed its levers, who also wrote the judge prompt. Measured: on the defects the key would hold without any reviewer's comment its lead holds, and in the earlier study scoring each reviewer only on defects some other source raised moved

no reviewer by more than 0.8 points [1]. Not done: a human audit of the key, and a key built before any reviewer ran.

No human has verified the key for the 145 new PRs. Claude agents built and critiqued it; severity grades, which drive the high-severity results, are likewise unchecked by a person.

The final runs are outside the key's pool. None of the final gpt-6.1-sol runs' comments were candidates for the key, so a real defect only they found is credited to nobody. The judge flags 61 possible such defects in QuickReview default's reviews and 4–49 in each other out-of-pool reviewer's (47 for the panel, 49 for open-code-review); per 100 comments three third-party tools have more than QuickReview's 6.1. These flags are unverified, and they could move the gaps in either direction by a few points [1].

One judge, sharing a model family. The final reviews were graded by gpt-6-sol only; the second judge, from another family, agreed on 98.7% of verdicts in the earlier study but did not grade these reviews. The judge shares a family with every reviewer here except Claude Code.

Development sets. Every September lever was designed from QuickReview's misses on the original 40 PRs; the other 145 are reported apart and show no in-sample advantage, though no lens was ablated on them. The frozen build's changes were designed from misses on the 145, on Martian's `tune20` and on Qodo's PRs, and validated on 20 of the original 40; the other 20, Martian's `test30` and 31 of Qodo's PRs were not used (Section 10).

Budget and inputs. QuickReview spends 2.6 to 21 times the cost of each third-party tool and reads the PR title and body, which four of the other tools did not in the final runs. We have not separated how much of the lead comes from the design and how much from the budget and the description (Section 7.4).

One run per reviewer. Every reviewer ran once. For QuickReview a full re-run reproduced the gpt-6-sol result and repeat runs bound its spread on 8 PRs. The competitors' configurations were chosen on gpt-6-sol and applied unchanged on gpt-6.1-sol.

Not the production configuration. The measured reviewer ran with the generic repository profile, no memory, no CI gate, and none of the production behaviour after a first review; the reviewer's code has changed since build `b2ce03a`, and the frozen build of Section 10 is the one that ships.

Nothing about people. The judge grades whether a comment is true and whether a defect was found. We did not measure whether developers act on the comments, how long the extra minor comments take to read, or what a false alarm costs a team.

Scope. Public open-source PRs of at most about 1,000 changed lines in ten languages, reviewed once each. Very large changes, private codebases and multi-round review are outside what was measured.

12 Conclusion

QuickReview is built for recall: it reviews a change through about ten narrow, independent lenses chosen by explicit rules, and posts only what a validator can re-derive from the code. On a benchmark that counts verified defects, with every competitor at its best documented

configuration and the OpenAI-based ones on the same model, its shipping build finds 97% of the high-severity defects and 89% of all of them, ahead of every competitor at every severity, and it has the highest recall on the real goldens of Martian's benchmark and on the functional bugs of Qodo's, ahead of the same three third-party harnesses on the same model on both. That recall has a price, stated with it: $2.06 and a median 17.5 minutes per PR, several times every competitor, and more comments, many of them true but minor, which costs it precision on both external benchmarks. The build on which the design was studied found 79% at $1.36 per PR, with the largest share of correct comments (95%); its extra spend over the closest competitor bought mainly low-severity coverage, and on high-severity defects pi-review's panel and Codex CLI were level with it at a fifth and a ninth of its cost per high-severity defect found.

The cost decomposition is the part of this report we would most want a designer of such systems to take away. Redundancy is what makes the design work — most defects are raised by several lenses, and the validator mostly collapses duplicates rather than rejecting claims — but its value is very uneven: a line-by-line sweep over every changed file earns its share many times over, while two of the code-quality lenses cost a tenth of the budget for almost nothing no other lens found. The misses that remain are recognition misses at the lenses, not filtering.

The closest competitor is also the closest design, several independent reviewers and a decision over what they raised. That is consistent with the view that independent angles plus a check carry the result, but every comparison that adds angles here also adds budget, and the model under the design moves recall from 28% to 84%. Whether this design beats a different way of spending the same budget is the question this report leaves open.

The frozen build (Section 10) is what closed the high-severity gap: lenses at the sites where base behaviour changes, instructions to exhaust a confirmed mechanism, a coverage check and higher effort on three passes took recall from 79% to 89%, at 51% more cost and 1.8 times the time.

References

[1] Mateusz Jakubczak. Measuring whether a code review finds the real defects: a verified defect benchmark on AACR-Bench, applied to eight reviewers at their best-shot configurations. Companion paper, 2026. `https://github.com/quickchatai/quickreview`, `bench/aacr/paper/benchmark.pdf`; its run data: `https://huggingface.co/datasets/quickchatai/quickreview-bench`.

[2] L. Zhang, Y. Yu, M. Yu, X. Guo, Z. Zhuang, G. Rong, D. Shao, H. Shen, H. Kuang, Z. Li, B. Wang, G. Zhang, B. Xiang and X. Xu. AACR-Bench: Evaluating automatic code review with holistic repository-level context. arXiv:2601.19494 [cs.SE], 2026 (v3, 30 January 2026). `https://arxiv.org/abs/2601.19494`

[3] Alibaba. Open Code Review (`@alibaba-group/open-code-review`), version 1.3.1 (npm, 10 June 2026); documentation at `main` (v1.12.11), Apache-2.0. `https://github.com/alibaba/open-code-review`

[4] OpenAI. Codex CLI (`@openai/codex`), versions 0.158.0 and 0.159.2, Apache-2.0. `https://github.com/openai/codex`

[5] OpenCode contributors. OpenCode (`opencode-ai`), version 1.18.33, MIT. Repository `anomalyco/opencode` (formerly `sst/opencode`). `https://github.com/anomalyco/opencode`

[6] Earendil Works. pi coding agent (`@earendil-works/pi-coding-agent`), versions 0.87.1 and 0.99.1, MIT. `https://github.com/earendil-works/pi`

[7] Z. Deng. pi-review (`@zephyrdeng/pi-review`), version 0.18.0, MIT. `https://github.com/ZephyrDeng/pi-review`

[8] Qodo and contributors. PR-Agent (`pr-agent` on PyPI), version 0.46.0, MIT. Repository `the-pr-agent/pr-agent` (formerly `qodo-ai/pr-agent`); documentation `https://docs.pr-agent.ai/`. `https://github.com/the-pr-agent/pr-agent`

[9] Anthropic. Claude Code (Anthropic's npm package), version 2.1.283. Documentation: "Code Review" (section "Review a diff locally") and "Model configuration" (effort levels), accessed 1 October 2026. `https://code.claude.com/docs/en/code-review`, `https://code.claude.com/docs/en/model-config`

[10] A. Bacchelli and C. Bird. Expectations, outcomes, and challenges of modern code review. In *Proc. 35th International Conference on Software Engineering (ICSE)*, pages 712–721, 2013. doi:10.1109/ICSE.2013.6606617

[11] M. V. Mäntylä and C. Lassenius. What types of defects are really discovered in code reviews? *IEEE Transactions on Software Engineering*, 35(3):430–448, 2009. doi:10.1109/TSE.2008.71

[12] C. Sadowski, E. Söderberg, L. Church, M. Sipko and A. Bacchelli. Modern code review: a case study at Google. In *Proc. 40th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 181–190, 2018. doi:10.1145/3183519.3183525

[13] V. R. Basili, S. Green, O. Laitenberger, F. Lanubile, F. Shull, S. Sørumgård and M. V. Zelkowitz. The empirical investigation of perspective-based reading. *Empirical Software Engineering*, 1(2):133–164, 1996. doi:10.1007/BF00368702

[14] M. Vijayvergiya, M. Salawa, I. Budiselić, D. Zheng, P. Lamblin, M. Ivanković, J. Carin, M. Lewko, J. Andonov, G. Petrović, D. Tarlow, P. Maniatis and R. Just. AI-assisted assessment of coding practices in modern code review. In *Proc. ACM International Conference on AI-Powered Software (AIware)*, pages 85–93, 2024. doi:10.1145/3664646.3665664. arXiv:2405.13565

[15] U. Cihan, V. Haratian, A. İçöz, M. K. Gül, Ö. Devran, E. F. Bayendur, B. M. Uçar and E. Tüzün. Automated code review in practice. In *Proc. 47th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 425–436, 2025. doi:10.1109/ICSE-SEIP66354.2025.00043. arXiv:2412.18531

[16] Z. Li, S. Lu, D. Guo, N. Duan, S. Jannu, G. Jenks, D. Majumder, J. Green, A. Svyatkovskiy, S. Fu and N. Sundaresan. Automating code review activities by large-scale pre-training. In *Proc. 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 1035–1047, 2022. doi:10.1145/3540250.3549081. arXiv:2203.09095

[17] X. Tang, K. Kim, Y. Song, C. Lothritz, B. Li, S. Ezzini, H. Tian, J. Klein and T. F. Bissyandé. CodeAgent: Autonomous communicative agents for code review. In *Proc. 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 11279–11313, 2024. doi:10.18653/v1/2024.emnlp-main.632. arXiv:2402.02172

[18] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery and D. Zhou. Self-consistency improves chain of thought reasoning in language models. In *Proc. International Conference on Learning Representations (ICLR)*, 2023. arXiv:2203.11171

[19] J. Li, Q. Zhang, Y. Yu, Q. Fu and D. Ye. More agents is all you need. *Transactions on Machine Learning Research*, 2024. arXiv:2402.05120

[20] B. Brown, J. Juravsky, R. Ehrlich, R. Clark, Q. V. Le, C. Ré and A. Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling. arXiv:2407.21787 [cs.LG], 2024.

[21] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse and J. Schulman. Training verifiers to solve math word problems. arXiv:2110.14168 [cs.LG], 2021.

[22] S. Dhuliawala, M. Komeili, J. Xu, R. Raileanu, X. Li, A. Celikyilmaz and J. Weston. Chain-of-verification reduces hallucination in large language models. arXiv:2309.11495 [cs.CL], 2023.

[23] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum and I. Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *Proc. 41st International Conference on Machine Learning (ICML)*, PMLR 235, pages 11733–11763, 2024. arXiv:2305.14325

[24] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song and D. Zhou. Large language models cannot self-correct reasoning yet. In *Proc. International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.01798

[25] J. C. Knight and N. G. Leveson. An experimental evaluation of the assumption of independence in multiversion programming. *IEEE Transactions on Software Engineering*, SE-12(1):96–109, 1986. doi:10.1109/TSE.1986.6312924

[26] S. Kapoor, B. Stroebl, Z. S. Siegel, N. Nadgir and A. Narayanan. AI agents that matter. *Transactions on Machine Learning Research (TMLR)*, 2025. arXiv:2407.01502

[27] L. Chen, M. Zaharia and J. Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. arXiv:2305.05176 [cs.LG], 2023.

A Tuning history in brief

The chronological experiment log (a working note) records the measurements that shaped the shipped default. We summarise the points relevant to judging overfitting; all are single runs on small sets on gpt-6-sol and are superseded by the full benchmark.

- Prompt iterations 1–5 did not measurably change defect-finding (every version 49–54.5 of 97 defects on 30 PRs); they raised AACR F1. Two runs of one build differed by 2.5 defects.
- Reasoning effort was the largest single lever measured before the levers existed: every model at maximum effort found 24.5 of 34 defects on smoke10 (one run) against 15.5–17.5 for eight normal-effort runs, at about three times the cost and time. It was not shipped as default.
- The missed-defect analysis behind the levers traced 48 defects the review kept missing on the 30 PRs: 42 were never raised by any pass, 15 were attention failures, 11 needed library knowledge. The levers (sub-lenses, sweep, dependency sources) raised found defects from an expected 30.5 (mean of seven earlier normal-effort runs) to 45.5 of 67 on the 16 in-sample PRs (one run), and from 20.5 to 25.0 of 29 on fresh10 (one run per arm) — about half the in-sample gain per PR.
- The lean plan kept recall at lower cost and became the `default` preset; `low` costs about half and gives back most of the levers' gain.
- The tests lens and the "every problem on the lines" rule raised fresh10 recall to 27 / 26.5 of 29 over two runs — by then fresh10 was in-sample.
- The code-quality lenses (runs qon and qoff, one review per PR) added 2 problems found on 9 PRs (mostly low severity, within noise) and raised AACR maintainability matches from 1 to 2 of 30, at +28% cost; abstraction posted nothing on its own there, the first sign of what Table 8 later showed.

B Chronology of result-affecting decisions

Table 17: Chronology (2026). "Tuned on" says which PRs informed each decision.

When	What	Tuned on
09-25	Harness wrapper, OCR arm, smoke10/holdout10/dev10, reference labels (agent-produced)	—
09-25/26	Review iterations 1–2 (validator premises, sibling sites, judgment lenses)	smoke10 AACR misses (before the defect benchmark existed)
09-26	Defect judge; ground truth v1/v2 for the 30	—
09-26	Levers: effort, sub-lenses, sweep, dependency sources, restate-not-kill	missed defects on the 30 (holdout10 stops being a holdout)
09-26	fresh10 created as out-of-sample check; its truth v1	—
09-27	Lean plan; presets default/low/max	16 of the 30 + fresh10 cost/recall
09-27	Tests lens; every-problem-on-the-lines	high-severity misses on fresh10 + the 30 (fresh10 used for design from here)
09-27	Code-quality lenses	9 PRs (3 fresh10 + 6 of the 30), both metrics
09-27	Validator demote-not-kill, `[review] nits`, pass-consistency	3 fresh10 PRs
09-27 23:34	**rest145 created**; default/low/OCR runs start on `1ad710b`	— (no reviewer change after this)
09-28	Four new arms, Claude prose parser, key extension (+15), re-judging, second judge	— (no reviewer behaviour change)
09-28/29	Optional validator with a doubting veto, stack-neutral frozen passes, style guide from directive files	— (repository-setup work)
09-29/30	Current-build re-run of default (`9106a64`), stability repeats, base-model arms	— (measurement only)
09-30	Fairness audit; 40-PR selection set fixed by rule; every documented variant run once on gpt-6-sol; picks by the pre-set rule	— (competitor configurations; QuickReview unchanged)
09-30/10-01	Every OpenAI-based contender on gpt-6.1-sol; picks re-run unchanged on all 185 PRs	— (measurement only)